

Building BarsN and BarsP from source

An operating-system-independent guide

July 24, 2026

1 The one idea to remember

BARS ships C source that must be compiled into two products per method: a stand-alone *executable* (`barsN.out`, `barsP.out`) and a *shared library* (`barsN.so`, `barsP.so`). Compiled files are tied to one operating system *and* one CPU architecture. The portable rule, true on every platform, is therefore:

Copy the sources, not the binaries. Rebuild on each machine.

Never reuse a `*.o`, `*.so`, or `*.out` produced on a different machine: an object of the wrong architecture loaded by R is exactly the “incompatible architecture” / “found ... required ...” error. The same source tree, compiled locally, works everywhere.

2 What is platform-specific and what is not

Almost every correction the code needs is about *modern C compilers and portability*, not about a particular CPU or OS. Only two things truly vary by platform: the toolchain you install, and the flags that name the system BLAS/LAPACK.

Aspect	Same everywhere?	Note
<code>malloc.h</code> → <code>stdlib.h</code> edit	yes	portability fix
Relaxed compiler flags	yes	needed by clang 15+ / gcc 14+
Two build products (<code>.out</code> , <code>.so</code>)	yes	<code>make</code> + R CMD SHLIB
File layout (<code>exec/</code> , <code>libs/</code>)	yes	package convention
C compiler	no	clang / gcc / Rtools
BLAS & LAPACK link flags	no	resolved via R CMD config
CPU architecture of output	no	must match your R build

The trick that makes the build portable is to stop hard-coding library names and let **R itself** report the correct BLAS/LAPACK/compiler for the current machine (Section 5).

3 Step 0 — Prerequisites

You need a C compiler and a build of R whose architecture matches the machine.

- **macOS:** Xcode Command Line Tools — `xcode-select -install` (provides clang).
- **Linux:** the compiler and R headers, e.g. Debian/Ubuntu `sudo apt-get install build-essential r-base-dev`; Fedora/RHEL `sudo dnf install gcc make R-devel`.

- **Windows:** **Rtools** matching your R version, on the build PATH.

Confirm the compiler and R agree on architecture before building:

```
cc --version      # or: gcc --version
R --version       # architecture here must match the compiler
```

On Apple Silicon, R `-version` must report `aarch64`; if it says `x86_64` you are running the Intel build under Rosetta and the output will be built for the wrong architecture.

4 Step 1 — Bring the corrected sources across

Copy the `barsN` and `barsP` source folders. **First remove any compiled products that travelled with them**, since they belong to another machine:

```
cd barsN && rm -f *.o *.so *.out
cd ../barsP && rm -f *.o *.so *.out
```

If you are starting from pristine upstream sources, apply the one portability edit that BARS needs — `<malloc.h>` does not exist on all systems, and `<stdlib.h>` provides the same declarations portably:

```
# in both barsN/ and barsP/, in logspline.c (line 3):
#   change   #include <malloc.h>
#   to       #include <stdlib.h>
```

This edit is correct on every operating system (it is required on macOS and harmless on Linux and Windows).

5 Step 2 — Build both products, portably

Two separate builds are required for each method:

- the **executable** `barsN.out` — this is what the package’s R code actually runs, via `system()`;
- the **shared library** `barsN.so` — placed in the package’s `libs/` directory per its layout.

Rather than hard-code `-llapack -lblas` (and the obsolete `-lg2c`, which no modern system has), we ask R for the flags it was built with. These commands are **identical on macOS, Linux, and Windows**; only R’s answers differ underneath.

BarsN

```
cd barsN
CC=$(R CMD config CC)
LIBS="$(R CMD config LAPACK_LIBS) $(R CMD config BLAS_LIBS) $(R CMD config FLIBS)"
FLAGS="-std=gnu89 -Wno-implicit-function-declaration -Wno-implicit-int"

# (a) the executable
$CC $FLAGS -o barsN.out \
    barsN.c barsN_funcs.c barsN_utils.c logspline.c ranlibsub.c com.c $LIBS

# (b) the shared library
PKG_CFLAGS="$FLAGS" PKG_LIBS="$LIBS" \
R CMD SHLIB -o barsN.so \
```

```
barsN.c barsN_funcs.c barsN_utils.c logspline.c ranlibsub.c com.c
```

BarsP

Identical, with P for N (ensure the `malloc.h` edit is present first):

```
cd ../barsP
$CC $FLAGS -o barsP.out \
    barsP.c barsP_funcs.c barsP_utils.c logspline.c ranlibsub.c com.c $LIBS

PKG_CFLAGS="$FLAGS" PKG_LIBS="$LIBS" \
R CMD SHLIB -o barsP.so \
    barsP.c barsP_funcs.c barsP_utils.c logspline.c ranlibsub.c com.c
```

The `-std=gnu89` and `-Wno-implicit-*` flags let this legacy C89 code compile under modern clang and gcc, which now treat implicit declarations and implicit `int` as errors by default. Expect only harmless `-Wformat-security` and `unused-variable` warnings.

What R CMD config resolves to. On macOS this expands to Apple’s Accelerate framework; on Linux to R’s own `libRblas/liblapack`; on Windows to the libraries bundled with Rtools. You do not need to know which — that is the point of asking R.

6 Step 3 — Confirm the architecture

The check that matters most: executable, library, and R must all be the same architecture.

```
file barsN.out barsN.so barsP.out barsP.so
```

On macOS each line should report `arm64` (Apple Silicon) or `x86_64` (Intel), matching your R; on Linux `x86-64` or `aarch64`. If any product reports the wrong architecture it is a foreign copy that slipped through: `rm -f *.o *.so *.out` and rebuild.

7 Step 4 — Place the products in the package

Put the executables in `exec/` and the shared libraries in `libs/`:

```
cp barsN/barsN.out barsP/barsP.out BARS/exec/
cp barsN/barsN.so barsP/barsP.so BARS/libs/
```

The package is then ready to install (or re-package as a source tarball with `R CMD build BARS`; remember such a tarball should carry *source*, so each target machine rebuilds its own binaries).

8 The ~/.R/Makevars caveat

If you carry over a personal `~/.R/Makevars` from another machine, any line naming a machine-specific compiler or library path (a hard-coded `gfortran` location in `FLIBS`, for instance) will be wrong on the new machine and can derail unrelated source installs. Either delete such lines and let R use its own defaults, or repoint them at the new machine’s toolchain. Architecture-independent definitions (such as `-DCalloc=R_Calloc -DFree=R_Free -DRealloc=R_Realloc`) are safe to keep.

One-line summary

Copy only the C sources; apply the `malloc.h`→`stdlib.h` edit; build `.out` with the compiler and `.so` with `R CMD SHLIB`, taking BLAS/LAPACK flags from `R CMD config`; verify with `file`; then drop `.out` into `exec/` and `.so` into `libs/`. The procedure is the same on every operating system — only the toolchain you installed differs.